

ECM3412/ECMM406

Nature Inspired Computation

Lecture 4

Evolutionary Algorithms in Detail
Selection Schemes, Operators and
Representations/Encodings

Today

- Algorithm Types
- Selection Operators & Issues
- Replacement Operators
- Representations/Encodings
- Mutation Operators
- Crossover Operators

Some Terms Used in Evolutionary Computation

- **Algorithm Type**
 - Generational
 - Steady State
 - Elitist
- **Selection**
 - Rank-Based
 - Roulette Wheel
 - Tournament
- **Replacer**
 - Weakest
 - First weaker
- **Recombination (Crossover)**
 - Single-point
 - Multi-point
 - Uniform
- **Mutation**
 - Single Gene
 - Multiple Gene
- **Parameters**
 - Population size
 - Generations/iterations
 - Mutation rate
 - Crossover rate
 - Tournament size....
 - Etc

2 Types of Genetic Algorithm

- **Generational** – genetic operators applied repeatedly to generate new population.
- Generational **elitist** algorithms copy n **best** individuals **unchanged** to the next generation
- New solutions in **purple**.

Initial Pop

Name	F(S)
S1	0.1
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S10	0.1

Apply
selection
& genetic
operators 10x
to give new
population

Generation 1

Name	F(S)
S11	0.5
S12	0.3
S13	0.3
S14	0.7
S15	0.7
S16	0.9
S17	0.4
S18	0.9
S19	0.9
S20	0.3

Apply
selection
& genetic
operators 10x
to give new
population

Generation 2

Name	F(S)
S21	0.7
S22	0.8
S23	0.9
S24	0.9
S25	0.7
S26	0.8
S27	0.5
S28	0.7
S29	0.6
S30	0.7

2 Types of Genetic Algorithm II

- **Steady State** – genetic operators applied N times (N small, 1 or 2) and bad solutions replaced
- New solutions in purple.

Initial Pop

Name	F(S)
S1	0.1
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S10	0.1

Apply selection
& genetic
operators N
times to give
new
individuals

Name	F(S)
S11	0.1
S12	0.5

Replace weak solutions in
population

Generation 1

Name	F(S)
S12	0.5
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S11	0.1

Replacement I

- A few different strategies exist for replacement in **steady-state** algorithms:
- Replace Weakest:

Initial Pop

Name	F(S)
S1	0.1
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S10	0.1

Apply
selection
& genetic
operators N
times to give
new
individuals

Name	F(S)
S11	0.1
S12	0.5

Replace weak solutions in
population

Generation 1

Name	F(S)
S12	0.5
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S11	0.1

Replacement II

- Replace First Weakest:

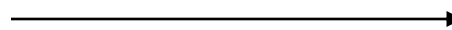
Initial Pop

Name	F(S)
S1	0.3
S2	0.5
S3	0.3
S4	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S10	0.1

New Solutions

Name	F(S)
S11	0.2
S12	0.5

Find first solution which is weaker than the new solution



Generation 1

Name	F(S)
S12	0.5
S2	0.5
S3	0.3
S11	0.2
S5	0.9
S6	0.7
S7	0.3
S8	0.4
S9	0.4
S10	0.1

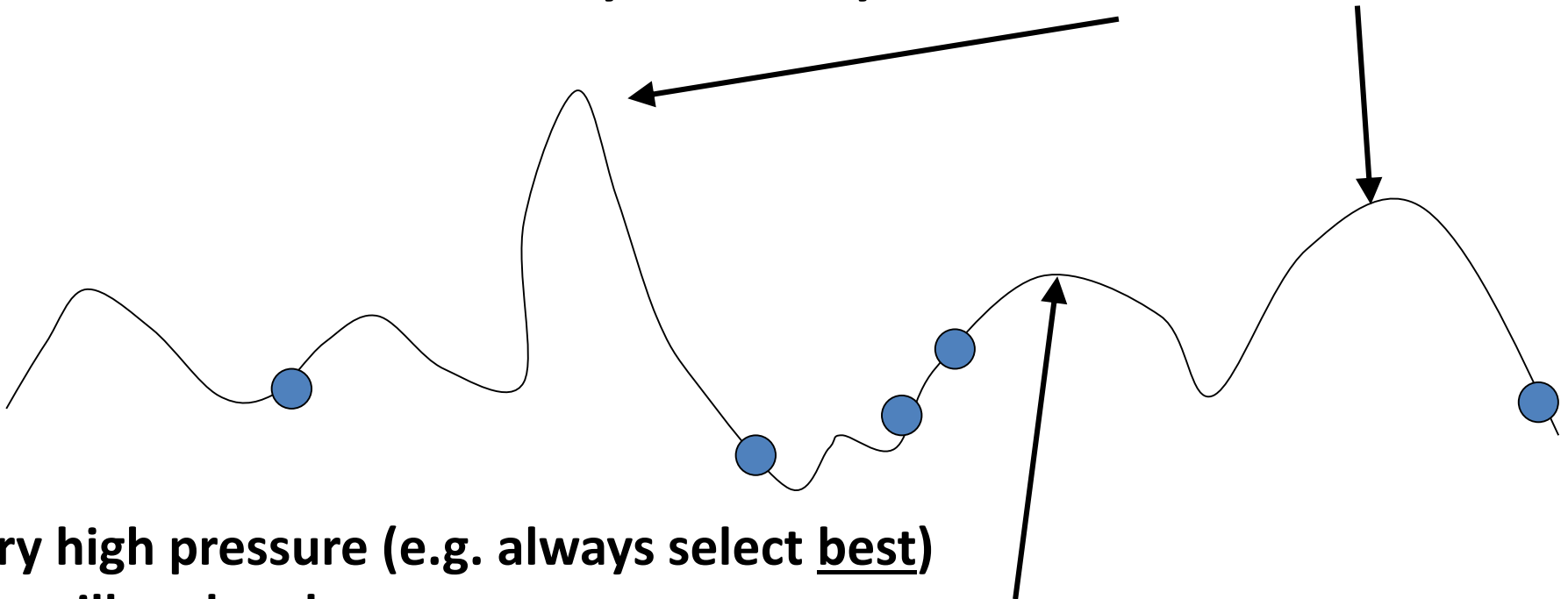
Selection Issues

Very low selection pressure (e.g. random)

No evolutionary 'progress' at all.

A modest level of pressure.

You may well find yourself here or here:



Very high pressure (e.g. always select best)
You will end up here:

Some Selection Methods

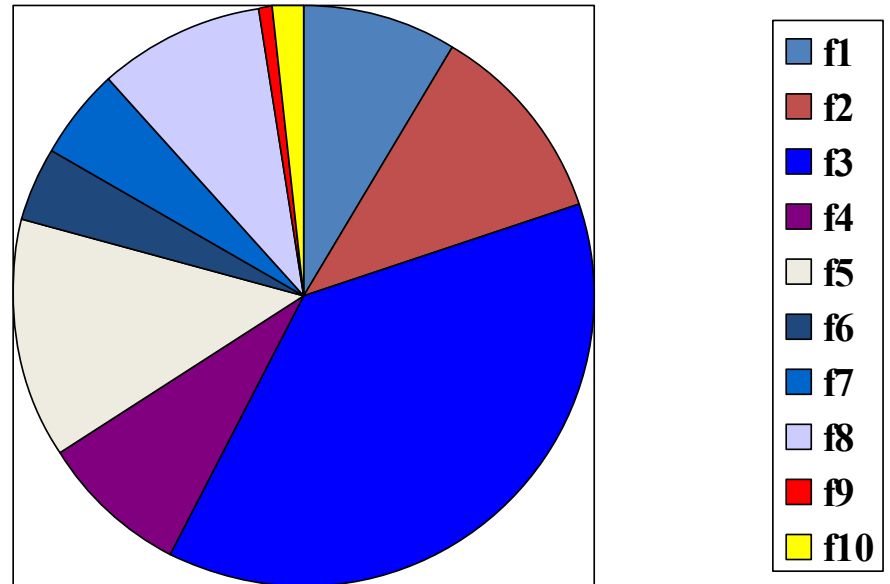
Grand old method:

Fitness Proportionate Selection also called
Roulette Wheel selection

Suppose there are P individuals with fitnesses $f_1, f_2, \dots, f_P$
The probability of selecting individual i is simply:

$$\frac{f_i}{\sum_{k=1}^P f_k}$$

*This is equivalent to spinning
a roulette wheel with sectors
proportional to fitness*



Problems with Roulette Wheel Selection

Having prob of selection directly proportional to selection seems fair.

It is still used a lot, **but**:

What about when we are trying to *minimise* the 'fitness' value?

What about when we may have negative fitness values?

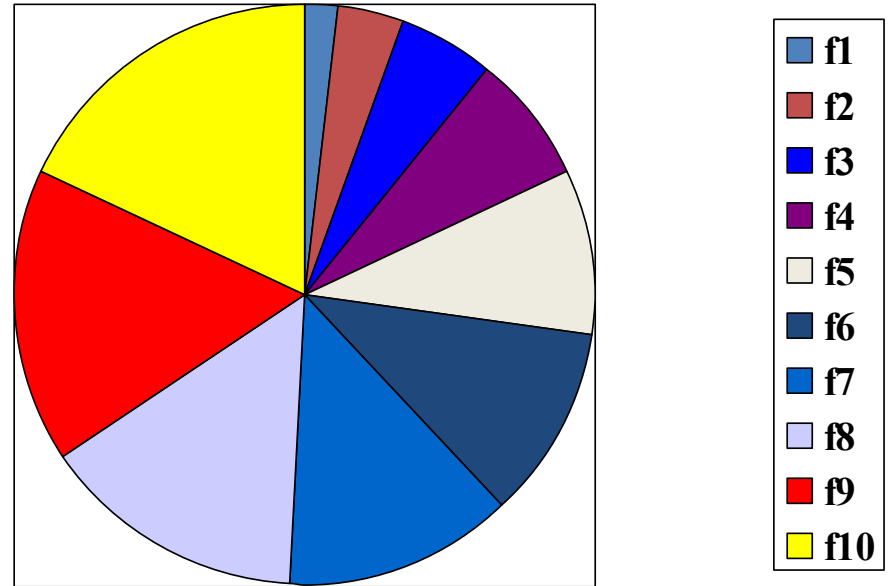
- Suppose we are trying to maximise a function, and we have a population of 5 fitnesses:
100, 0.4, 0.3, 0.2, 0.1 -- the best is 100 times more likely to be selected than all the rest put together!
- A modified $f(s)$ adding 100 to all fitnesses might give us:
200, 100.4, 100.3, 100.2, 100.1 – a much more reasonable situation.
- Take home message: Fitprop requires us to be **very careful** how we design the fine detail of fitness assignment.
- Other selection methods are better in this respect, and more used now.

Rank Based Selection

The fitnesses in the pop are
Ranked from Popsiz
(fittest) down to 1 (least fit),
when maximising.

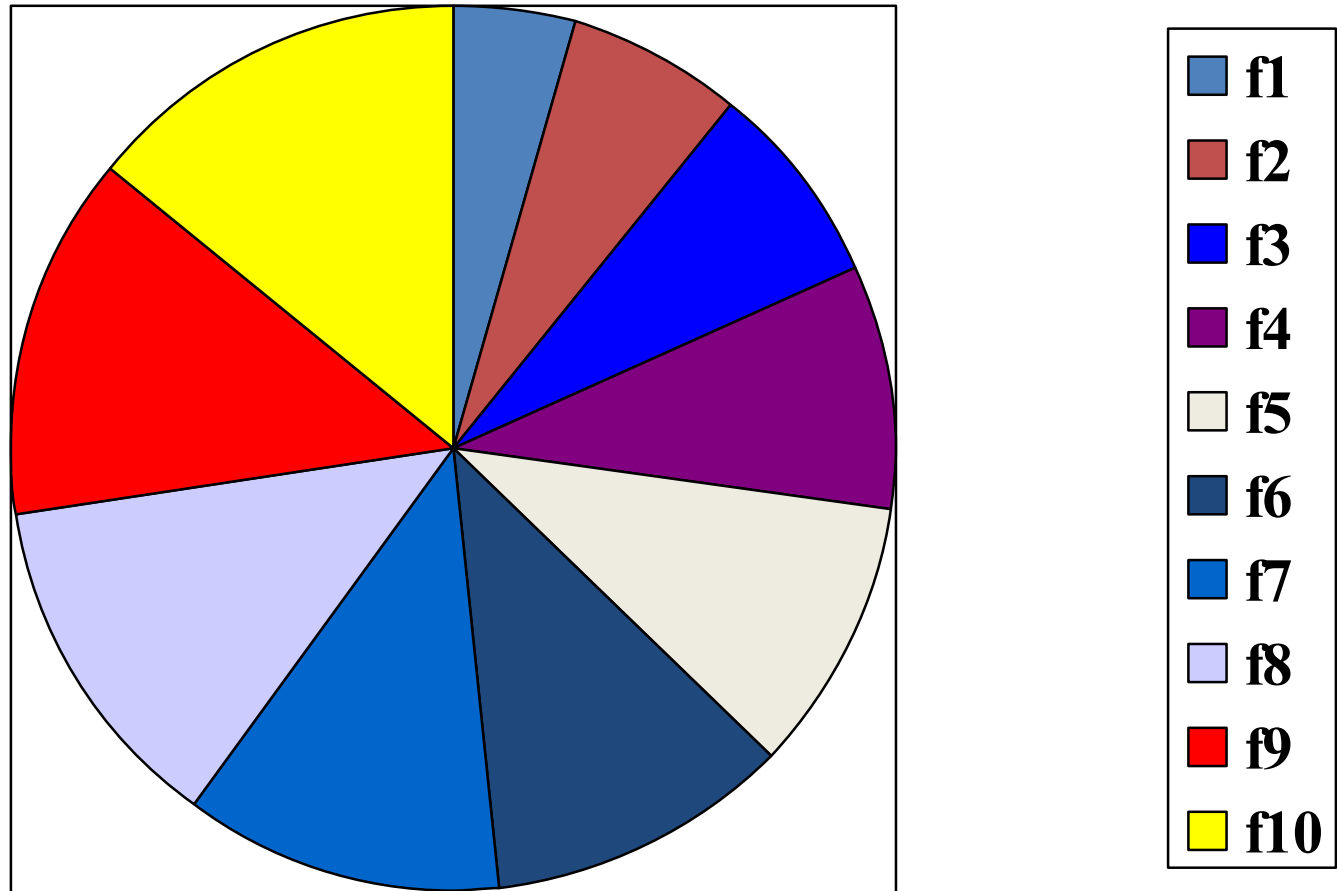
The selection probabilities are
proportional to **rank**.

There are variants where the
selection probabilities are a function
of the rank.



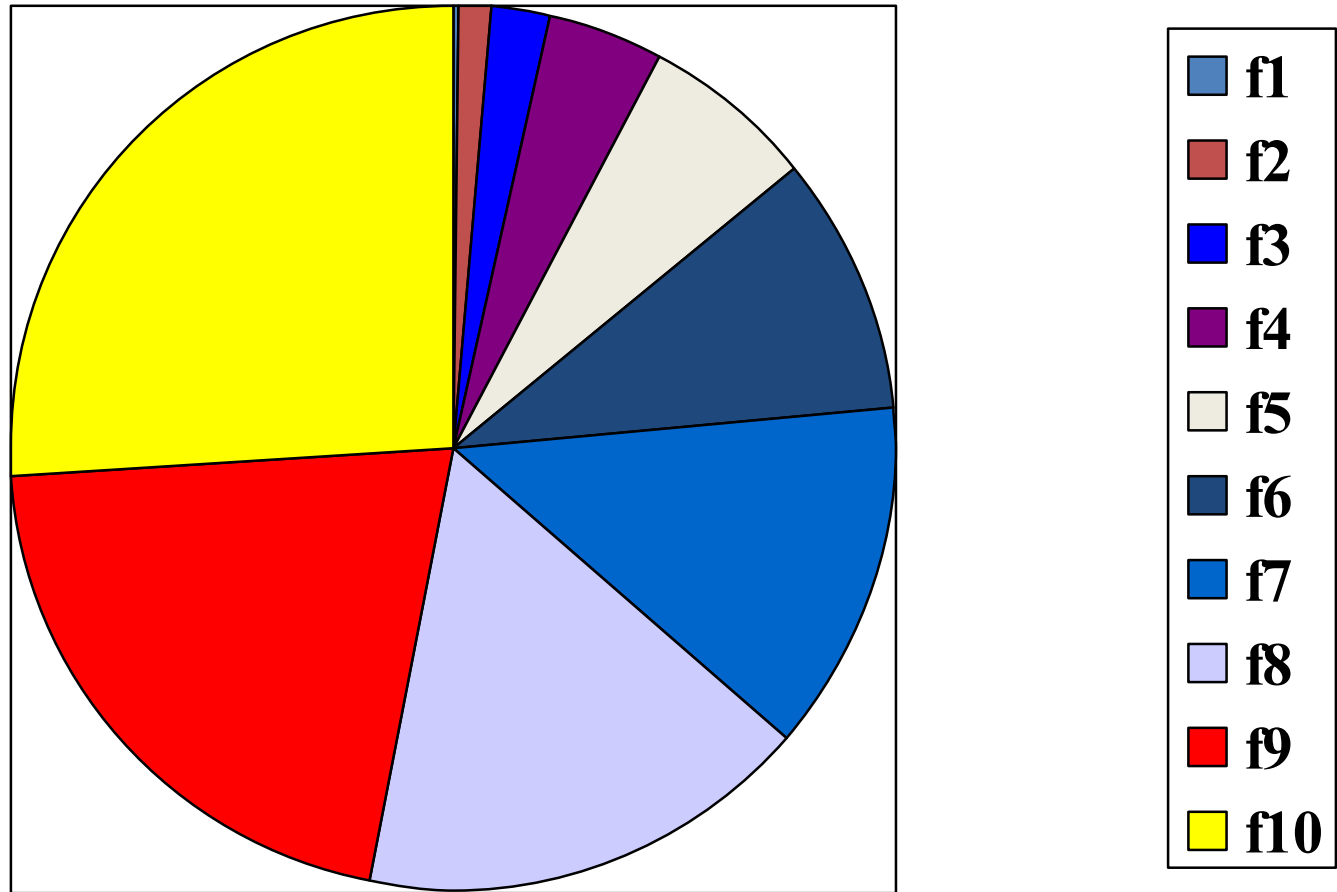
Rank with low bias

Here, selective fitnesses are based on $\text{rank}^{0.5}$



Rank with high bias

Here, selective fitnesses are based on *rank*²



Tournament Selection

Tournament selection: tournament size = t

Repeat t times

choose a random individual from the pop
and remember its fitness

Return the best of these t individuals (BTR)

- + Tunable
- + Avoids the problems of superfit or superpoor solutions
- + Very simple to implement and very efficient (**does not require pop sorting**)
- Requires tuning another parameter (tournament size)

Tournament Selection

Parameter: tournament size, t

To select a parent, randomly choose t individuals from the population (with replacement).

Return the fittest of these t

What happens to **selection pressure** as we increase t ?

What degree of selection pressure is there if $t = 10$ and $popsiz = 10,000$?

How to think of genetic operators

- ***Selection*** represents our strategy for deciding which areas of the search space to focus our efforts on.
- ***Genetic Operators*** provide our means to generate new candidate solutions.
 - ***Exploitation***: We want operators to have a fair chance of yielding good new solutions (small change, and/or combine bits from solutions we already know are good)
 - ***Exploration***: We also (obviously) want to be able to potentially explore the whole space.

Representation/Encoding

- The chromosome of a solution can be (for example):
 - Integer Vectors
 - Binary Strings
 - Real Values
 - Permutations
 - Trees
- Typically, different representations require representation-specific search operators to work effectively or at all.

K-ary Encoding

In a ***k*-ary encoding** ($k=2$, binary), a candidate solution is just a list of ***L* numbers**, each of which can be anything from 0 to $k-1$ (or 1 to k).

These might be candidate solutions from a k -ary encoding with $L = 10$ and $k = 20$:

[17, 2, 19, 1, 1, 1, 5, 11, 12, 2]

[16, 19, 2, 19, 2, 3, 4, 7, 5, 2]

E.g. for the Water Distribution Problem: a simple representation for 100 pipes and 5 possible diameters, would be a 5-ary encoding of length $L=100$.

Mutation in k -ary encodings

Single-gene mutation:

Choose a gene at random, and change it to a random new value. E.g. 352872 $\rightarrow$ 312872

M -gene mutation:

Multiple instances of single-gene mutation

Swap mutation:

Choose two genes at random, and swap them. E.g. 352872 $\rightarrow$ 372852

Why is this probably not very good in this context?

Mutation in Real-Valued Encodings

Often a candidate solution is a vector of real numbers of length L

E.g., (0.3, 0.2, 0.4, 0.2, 0.1)

Single-gene mutation:

Choose a gene at random, and add a small random deviation to it. Often chosen from a Gaussian distribution.

Vector mutation:

Generate a small random vector of length L , and add it to the candidate solution.

Mutation in Permutation-Based Encodings

Here is a permutation of length 10:

DEGJACBFIH

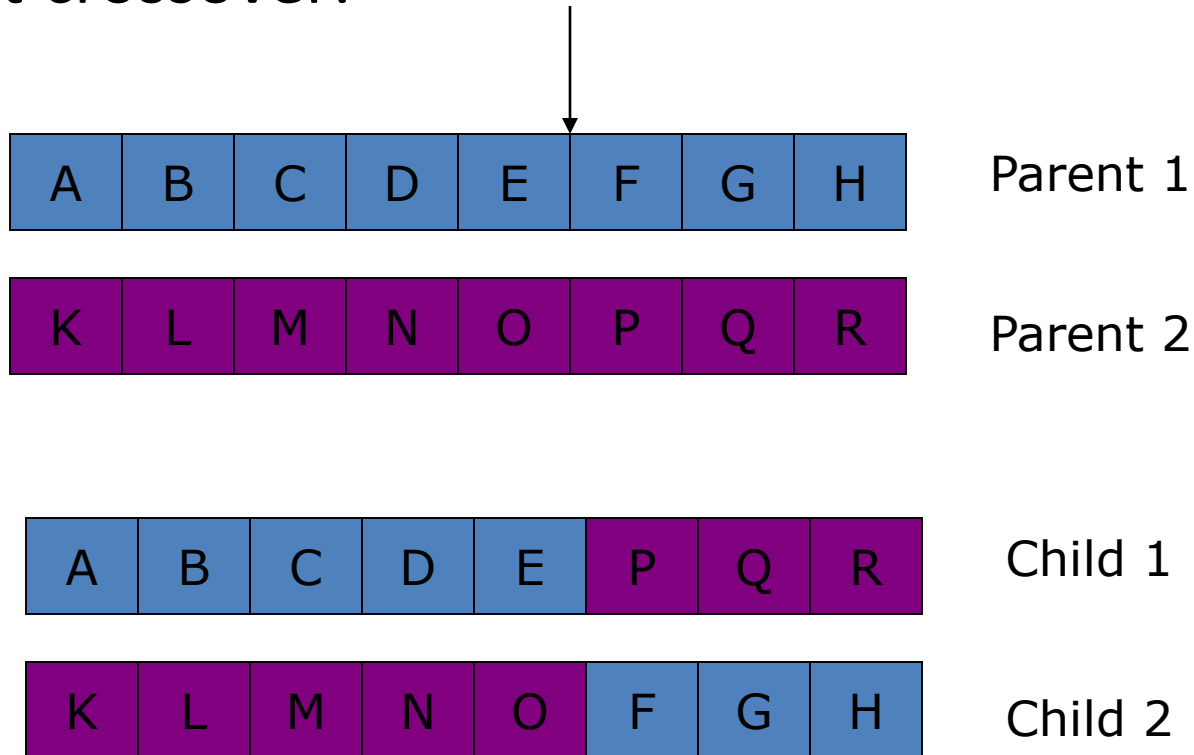
Can we do single-gene mutation?

Can we do swap mutation?

Recombination Methods for k -ary Encodings

The standard recombination operators are:

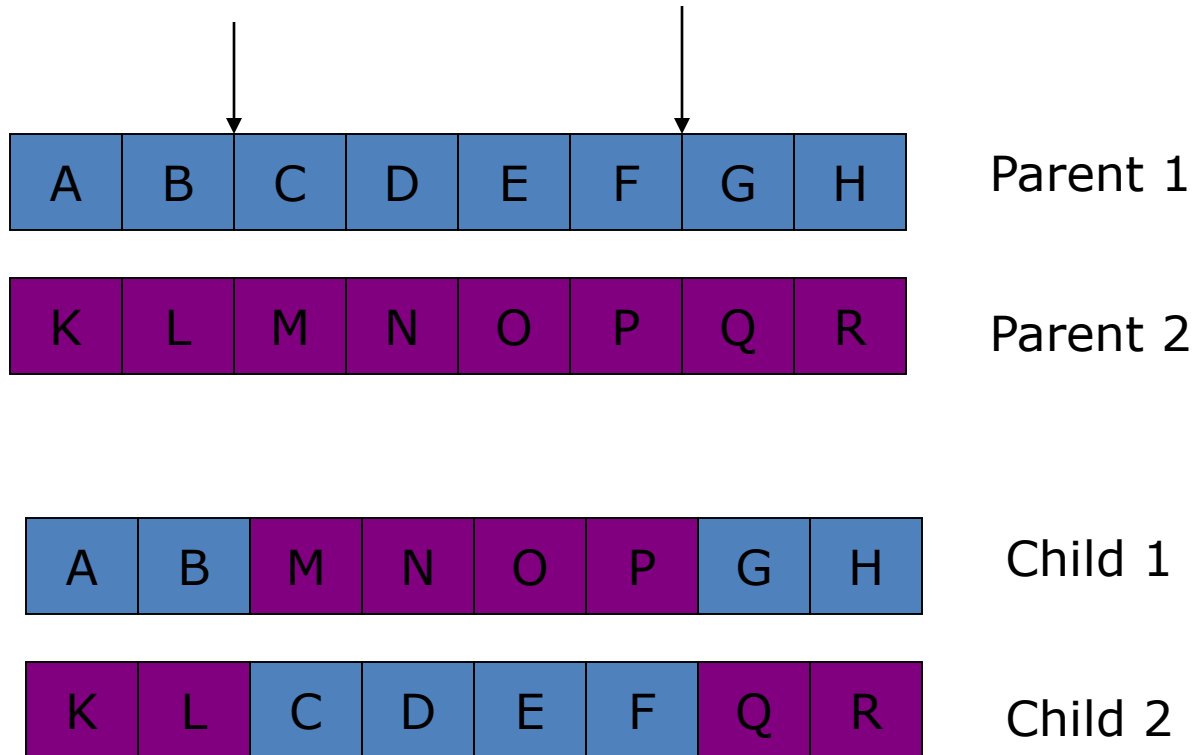
1-point crossover:



Recombination Methods for k -ary Encodings

2-point crossover:

k -point crossover – again, what do you think it is?



Uniform Crossover

Uniform Crossover

Generate a random binary 'mask'

Use it to decide which genes are swapped and which stay.

A	B	C	D	E	F	G	H
---	---	---	---	---	---	---	---

Parent 1

K	L	M	N	O	P	Q	R
---	---	---	---	---	---	---	---

Parent 2

0	1	0	0	1	1	0	1
---	---	---	---	---	---	---	---

Binary Mask

A	L	C	D	O	P	G	R
---	---	---	---	---	---	---	---

Child 1

K	B	M	N	E	F	Q	H
---	---	---	---	---	---	---	---

Child 2

Encoding / Representation

Maybe the main issue in (applying) EC

Note that:

- Given an optimisation problem to solve, we need to find a way of encoding candidate solutions
- There can be many very different encodings for the same problem
- Each way affects the shape of the landscape and the choice of best strategy for climbing that landscape.

A steady state, mutation-only, replace-worst EA with tournament selection

0. Initialise: generate a population of *popsiz*e random solutions, evaluate their fitnesses.
1. Run *Select* to obtain a parent solution *X*.
2. With probability *mutate_rate*, mutate a copy of *X* to obtain a mutant *M* (otherwise *M* = *X*)
3. Evaluate the fitness of *M*.
4. Let *W* be the current worst in the population. If *M* is not less fit than *W*, then replace *W* with *M*. (otherwise do nothing)
5. If a termination condition is met (e.g. we have done 10,000 evals) then stop. Otherwise go to 2.

Select: randomly choose *tsiz*e individuals from the population. Let *c* be the one with best fitness; return *X*.

A generational, elitist, crossover+mutation EA with Rank-Based selection

0. Initialise: generate a population G of *popsiz*e random solutions, evaluate their fitnesses.
1. Run *Select* $2 * (\text{popsiz}e - 1)$ times to obtain a collection I of $2 * (\text{popsiz}e - 1)$ parents.
2. Randomly pair up the parents in I (into $\text{popsiz}e - 1$ pairs) and apply *Vary* to produce a child from each pair. Let the set of children be C .
3. Evaluate the fitness of each child.
4. Keep the best in the population G and delete the rest.
5. Add all the children to G .
6. If a termination condition is met (e.g. we have done 100 or more generations (runs through steps 1–5) then stop. Otherwise go to 1,

A generational, elitist, crossover+mutation EA with Rank-Based selection, continued ...

Select: sort the contents of G from best to worst, assigning rank $popsiz$ to the best, $popsiz-1$ to the next best, etc ..., and rank 1 to the worst.

The ranks sum to $F = popsiz(popsiz+1)/2$

Associate a probability $Rank_i/F$ with each individual i .

Using these probabilities, choose one individual X , and return X .

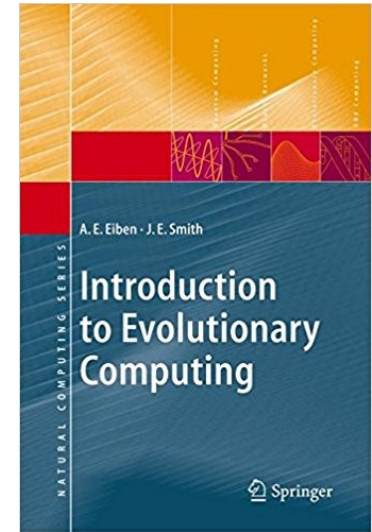
Vary:

1. With probability $cross_rate$, do a crossover:
I.e produce a child by applying a crossover operator to the two parents. Otherwise, let the child be a randomly chosen one of the parents.
2. Apply mutation to the child.
3. Return the mutated child.

Summary

- **Selection** should have a bias towards the fittest solutions
- There are different genetic **operators** for different **representations**
- Representation of the **problem** used in the algorithm and choice of genetic operators should be made together (design)

- Readings:
 - Introduction to Evolutionary Computing by Eiben & Smith
 - Ch 2 (components of EAs),
 - Ch 3 (representations & operators)



- Next Lecture:
more on representations/encodings and
search operators design